

Court IQ

Phase 03: Working AI Model Prof. Sitaram Ayyagari– ITAI-2277

NBA Player Performance Prediction Engine





LAL





Projection

PTS model: GradientBoosting

PTS	REB	AST	PRA
29.57	4.36	5.02	38.95

Confidence (Consistency)

61%

Consistency score from last N games volatility (not guaranteed).



Week 7: Model Architecture Design

For Week 7, we selected Gradient Boosting as our final model and designed the architecture to predict player stats using recent game data. This model improves accuracy by combining multiple decision trees, where each new model learns from previous errors.

```
PTS_NAME = "real_best_pts_gradient_boosting.pkl" # Points model
REB_NAME = "real_best_reb_gradient_boosting.pkl" # Rebounds model
AST_NAME = "real_best_ast_gradient_boosting.pkl" # Assists model
FEATURES_NAME = "feature_columns.json" # Feature structure used during training
```

This code shows how we load our trained Gradient Boosting models using joblib. Each model predicts a specific stat and is applied to processed game data to generate predictions.

```
# Load trained gradient boosting models
pts_model = joblib.load(CACHE_DIR / PTS_NAME)
reb_model = joblib.load(CACHE_DIR / REB_NAME)
ast_model = joblib.load(CACHE_DIR / AST_NAME)
```

Week 7: Frameworks, Tools, Resources



- Python
- Pandas / NumPy
- Scikit-learn
- Joblib
- Streamlit



```
$ pip install streamlit  
$ streamlit hello
```



- NBA API
- GitHub for version control
- Local machine for model development
- Cloud deployment through Streamlit



POINTS

1. Luka Dončić LAL	42
2. Devin Booker PHX	34
3. Jalen Duren DET	31
4. Jrue Holiday POR	30
5. Deni Avdija POR	28

REBOUNDS

1. Moussa Cisse DAL	13
2. Wendell Carter Jr. ORL	12
2. Moussa Diabaté CHA	12
4. Deni Avdija POR	11
5. Ousmane Dieng MIL	10

ASSISTS

1. Scottie Barnes TOR	12
1. Luka Dončić LAL	12
1. Jamal Shead TOR	12
4. Alperen Sengun HOU	10
5. LaMelo Ball CHA	9



In this slide, we show the tools, frameworks, and computing resources we used to build and run the model.

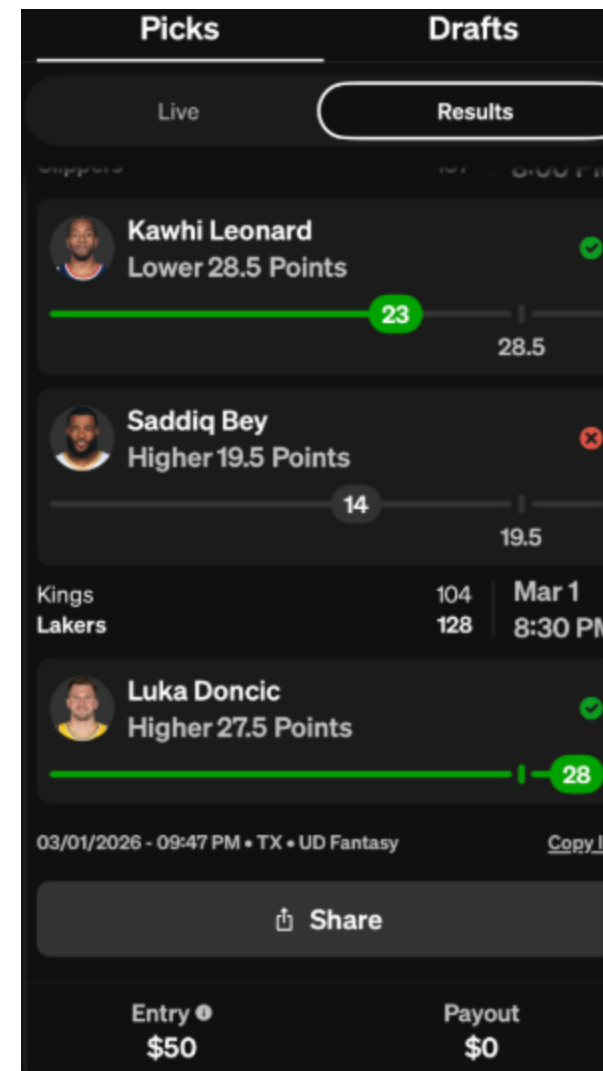
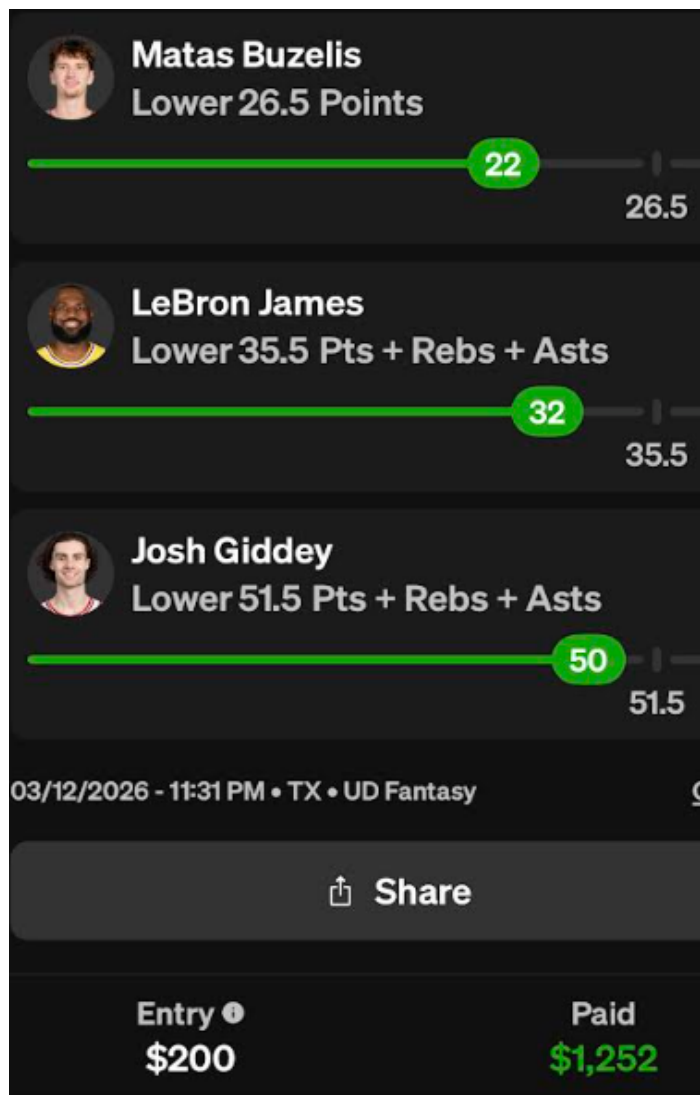
Week 8: Model Training and Optimization

In Week 8, we trained our Gradient Boosting model using NBA game data, tuned key hyperparameters, and improved performance through testing, debugging, and evaluation.

- Initial training of Gradient Boosting model on NBA game log data
- Performance monitoring during training
- Debugging prediction pipeline
- Iterative testing and model comparison
- Model improvement based on results

Hyperparameter tuning:

- `n_estimators`
- `learning_rate`
- `max_depth`



Week 9: Evaluation and Iteration

Evaluation metrics:

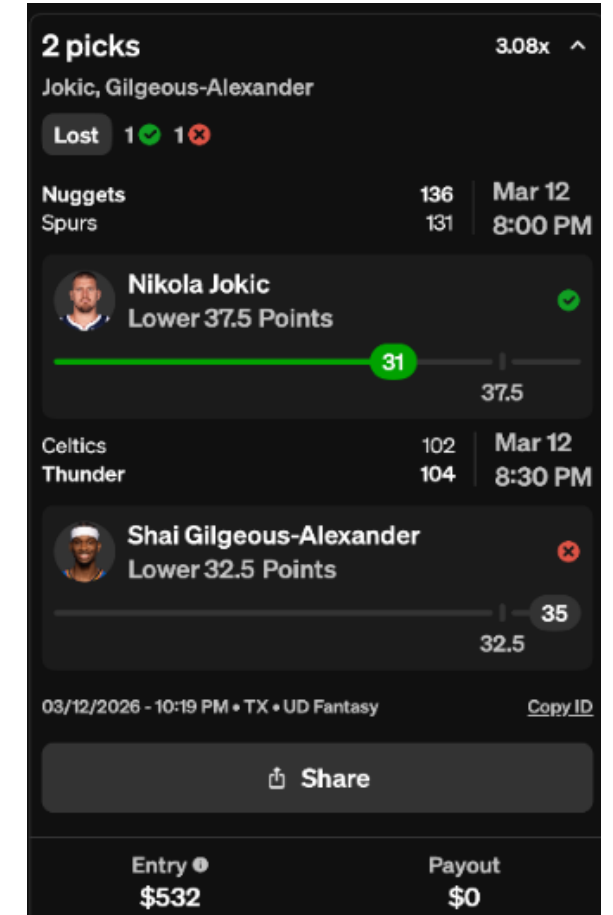
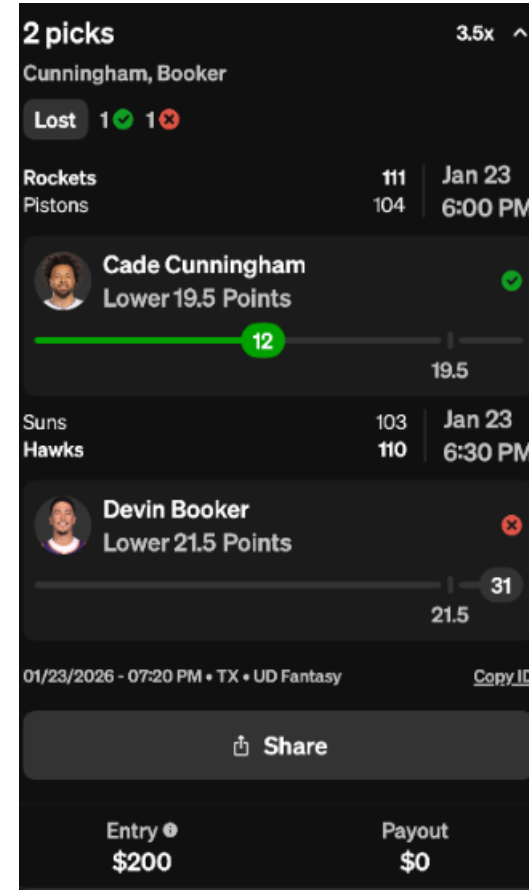
- MAE (Mean Absolute Error)
- RMSE (Root Mean Squared Error)

Example results (PTS model):

- MAE ≈ 2.24
- RMSE ≈ 3.32

- Model comparisons:
 - Linear Regression
 - Random Forest
 - Gradient Boosting
- Error analysis on high-variance players
- Performance improvements through feature refinement

Gradient Boosting selected as final model



This slide explains Week 9, where we evaluated the model, compared results, analyzed errors, and made improvements based on what we found.

Week 10: Refinement and Interpretability

Model Refinement (Gradient Boosting):

- Final optimization and tuning of Gradient Boosting model for improved accuracy
- Feature importance used for interpretability

Feature Engineering Improvements:

- Added opponent-based features
- Added matchup adjustments
- Added confidence scoring
- Optional minutes-based architecture

Ensemble methods improve performance by combining predictions from multiple models, reducing errors and increasing accuracy. But we did not use them due to time constraints.

In Week 10, we refined our Gradient Boosting model by optimizing performance and improving interpretability using feature importance. We also enhanced the model with additional features and evaluated ensemble methods for future improvements.

src/courtiq/models/model_loader.py

Model Loading (Supervised Machine Learning)

This file loads pre-trained supervised learning models so they can be used for real-time predictions.

Content:

- Loads trained machine learning models from cloud storage
- Models are saved as .pkl files
- Uses `joblib` to load models into memory

Model Type:

- Gradient Boosting Regressor
- Supervised Learning

What's happening:

- Models were trained on historical NBA data
- Each model learned patterns between player stats and real outcomes



src/courtiq/models/predict.py

Prediction Engine (Gradient Boosting in Action)

The model learns from past mistakes to make more accurate predictions.

Content:

- Takes last N games of a player
- Converts stats into input features (averages)
- Sends features into trained models
- Models output predictions:
 - Points
 - Rebounds
 - Assists

How Gradient Boosting works:

- Builds many small decision trees
- Each tree learns from previous errors
- Predictions improve step-by-step



Final Deliverable

Prepared Model Deliverable

Prepared Model Deliverables:

- Trained .pkl model files
- feature_columns.json
- Evaluation report
- Streamlit deployment
- GitHub repository
- Documentation for architecture, training, evaluation, and deployment

greglivin-court-iq-production-app-dev-stpjhs.streamlit.app

github.com/GregLivin/Court-IQ-Production

Conclusion

Successfully built and deployed a machine learning regression model

Completed full machine learning pipeline:

- Model architecture design
- Model training
- Model evaluation
- Model refinement
- Deployment and delivery